

RAG 解析流程验证测试文档

RAG Pipeline Parsing Validation Test Document

本文档用于全面验证RAG（检索增强生成）系统的PDF解析能力，
覆盖了实际业务场景中可能遇到的各种文档元素和边界情况。

文档版本	V1.0
创建日期	2026年7月24日
文档密级	内部公开
测试范围	短段落、长段落、图片(文字/表格/折线图/公式)、表格、长表格、跨页表格、空白段落、页眉

第一部分：段落文本测试

本部分测试RAG解析器对不同类型段落文本的处理能力，包括短段落、长段落、空白段落、以及包含特殊格式的段落。

1.1 短段落测试

Django的ORM（对象关系映射）允许开发者使用Python类来定义数据模型，无需编写SQL语句。

在Django中，URL路由通过urls.py文件的urlpatterns列表进行配置。

Django中间件是一个轻量级的插件系统，用于全局修改请求和响应对象。

Django的select_related方法使用SQL JOIN来一次性获取关联对象，有效解决N+1查询问题。

Django signals（信号）允许某些发送者通知一组接收者某个动作已经发生，常用于解耦业务逻辑，例如post_save信号在模型保存后自动触发。

RAG系统需要能够正确处理这些较短的段落，确保每个包含实际语义信息的段落都被独立解析和索引，不会因为段落过短而被合并或忽略，从而保证检索时能精准命中这些知识点。

1.2 长段落测试

Django是一个基于Python的高级Web框架，遵循MVT（Model-View-Template）架构模式，由Adrian Holovaty和Simon Willison于2003年创建，并于2005年正式开源。Django的设计哲学强调DRY（Don't Repeat Yourself）原则，旨在帮助开发者以最少的代码快速构建功能完备的Web应用。Django的核心组件包括：ORM（对象关系映射）层，它将Python类映射到数据库表，开发者无需编写原生SQL即可完成CRUD操作，支持SQLite、PostgreSQL、MySQL、Oracle等多种数据库后端；URL调度器（URL Dispatcher），通过正则表达式或路径转换器将URL映射到对应的视图函数或类；模板引擎（Template Engine），使用基于文本的模板语言（Django Template Language, DTL）将业务逻辑与展示分离；表单处理（Form Handling），提供Form和ModelForm类自动生成HTML表单并进行数据验证；认证系统（Authentication System），内置User模型、权限管理和会话机制；admin后台管理界面，可根据模型自动生成CRUD管理页面，极大地提升了开发效率。在Django的ORM中，QuerySet是延迟加载的（Lazy Loading），只有在真正需要数据时才会执行数据库查询，这种机制允许通过链式调用构建复杂的查询。常用的查询优化技术包括：select_related用于一对一和外键关系的预加载（JOIN查询），prefetch_related用于多对多和反向外键关系的预加载（二次查询），values和values_list用于只获取需要的字段，only和defer用于控制字段加载，bulk_create和bulk_update用于批量操作以减少数据库往返次数。Django的中间件（Middleware）系统是一个请求/响应处理链，每个中间件可以在请求到达视图前或响应返回客户端前进行预处理，常用于实现认证、CORS、CSRF防护、日志记录、限流等功能。Django还提供了强大的缓存框架，支持Memcached、Redis等后端，可通过cache_page装饰器、模板缓存或底层cache API实现多级缓存策略。对于异步处理，Django 3.1+版本引入了对async视图和async ORM的原生支持，结合

ASGI服务器（如Daphne、Uvicorn）可以处理WebSocket和长轮询等实时通信场景。Django RESTFramework（DRF）是构建RESTful API的事实标准扩展，提供序列化器（Serializer）、视图集（ViewSet）、路由（Router）、认证、权限和限流等完整功能，大幅简化了API开发流程。理解这些Django核心概念对于Web开发至关重要，也是技术面试中常考的知识点。

1.3 空白段落测试

以下是空白段落和段落间距测试：

Django models.py 中定义每个继承自 models.Model 的类都对应数据库中的一张表。

Django views.py 中的视图函数接收 HttpRequest 对象作为第一个参数，返回 HttpResponse。

Django settings.py 中的 INSTALLED_APPS 列表注册了项目使用的所有应用。

Django migrate 命令会根据 migrations 目录下的迁移文件同步数据库 schema。

注意：RAG解析器应当正确处理空白段落--不应将过大的空白视为文档结束，也不应将多个空白合并为一个。合理的空白间距意味着文档结构的变化。

1.4 混合中英文段落测试

Django是一个基于 Python 的高级 Web framework，遵循 MVT (Model-View-Template) 架构模式。在 Django 项目中，ORM (Object-Relational Mapping) 是最核心的 feature 之一，It allows developers to interact with the database using Python objects instead of raw SQL. Django 的 QuerySet API 支持丰富的 lookup methods，包括 filter()、exclude()、get()、order_by() 等。常用的 field types 有：CharField、TextField、IntegerField、DateTimeField、ForeignKey、ManyToManyField 等。在 Django REST Framework (DRF) 中，Serializer 负责将 model instances 转换为 JSON response，而 ViewSet 则提供了 list()、create()、retrieve()、update()、destroy() 等 standard CRUD actions。评估 Django 应用性能时，我们关注 metrics 如：response time (ms)、queries per request、database connection pool usage、cache hit rate。Django 支持 WSGI 和 ASGI 两种部署方式，production 环境常使用 Gunicorn (WSGI) 或 Uvicorn (ASGI) 配合 Nginx 做反向代理。

1.5 特殊字符与Unicode测试

特殊符号测试：© 版权符号 ® 注册商标 ™ 商标符号 € 欧元符号 ¥ 人民币符号 £ 英镑符号

数学符号：± 正负号 × 乘号 ÷ 除号 ≠ 不等于 ≤ 小于等于 ≥ 大于等于 ∞ 无穷

希腊字母：α (alpha) β (beta) γ (gamma) δ (delta) ε (epsilon) θ (theta) λ (lambda) π (pi)

上标数字: ¹ ² ³ ⁴

下标数字: ₁ ₂ ₃ ₄

箭头: ← ↑ → ↓ ↔ ⇒

方块: ■ ▲ ◆ ★ √

全角数字: 0 1 2 3 4 5 6 7 8 9

全角字母: A B C a b c

Emoji:

注意: RAG解析器需要正确处理这些特殊Unicode字符, 包括emoji。某些PDF解析器会丢失或错误转换这些字符。

1.6 多级段落层级测试 (4层以上)

以下是多级标题层级结构, 测试RAG解析器对文档层次结构的理解能力:

第一章 Django架构概述

本章介绍Django框架的整体架构设计, 包括各模块的功能划分和交互关系。

1.1 Model (模型) 层

Model层负责数据访问和业务逻辑, 通过ORM将Python类映射到数据库表。

1.1.1 ORM查询引擎

ORM查询引擎是Model层的核心组件, 负责将Python方法调用转换为SQL语句执行。

1.1.1.1 QuerySet延迟加载

QuerySet是延迟加载的, 只有在迭代、切片、len()或list()时才会真正执行数据库查询。这种机制允许链式调用构建复杂查询。

1.1.1.1.1 查询缓存机制

每个QuerySet内部维护一个_result_cache, 首次执行后结果会被缓存, 后续访问直接从缓存读取, 避免重复查询数据库。

1.1.1.1.2 查询优化策略

select_related用于外键和一对一关系的JOIN预加载, prefetch_related用于多对多和反向外键关系的二次查询预加载。

1.1.1.2 事务管理子模块

Django通过atomic()装饰器或上下文管理器提供事务管理, 支持savepoint和嵌套事务。默认使用数据库的自动提交模式。

1.1.2 数据库迁移引擎

makemigrations命令对比models定义与历史迁移文件，生成新的迁移文件；migrate命令执行迁移文件更新数据库schema。

1.2 View（视图）层

View层负责处理HTTP请求并返回响应，支持函数视图（FBV）和类视图（CBV）两种形式。

1.2.1 类视图继承体系

Django提供View、TemplateView、ListView、DetailView、CreateView、UpdateView、DeleteView等基础类视图，支持MRO继承复用。

1.2.2 装饰器与Mixins

常用装饰器包括login_required、permission_required、csrf_exempt；Mixins提供多继承方式组合功能，如LoginRequiredMixin。

第二章 Template与Form

本章介绍Django的模板引擎和表单处理机制。

2.1 模板系统配置

2.1.1 模板继承

2.1.1.1 extends与block标签

通过`{% extends 'base.html' %}`继承父模板，`{% block content %}{% endblock %}`定义可覆盖的区域，实现模板复用。

2.1.1.2 模板标签与过滤器

常用标签包括`{% for %}`、`{% if %}`、`{% url %}`、`{% csrf_token %}`；常用过滤器包括date、default、length、slice、safe等。

第二部分：图片内容测试

本部分测试RAG解析器对PDF中嵌入图片的处理能力。图片可能包含文字内容、表格、图表、公式等，解析器需要能够提取图片中的信息，或至少识别图片的存在并标注其位置。

2.1 图片中的文字内容

以下图片包含文字内容（模拟扫描件中嵌入的文字图片）。RAG解析器应当能够通过OCR识别图片中的文字，或者至少标注此处存在一张包含文字的图片。

关于2026年度技术研发工作总结报告

一、研发工作概述

2026年度，本部门在人工智能与大模型技术领域取得了显著进展。全年共完成研发项目12项，其中核心项目包括：大语言模型训练框架优化、多模态理解系统开发、以及RAG检索增强生成平台的搭建与迭代。团队规模从年初的15人扩充至年底的28人。

二、核心技术突破

- 成功实现了基于Transformer架构的模型压缩技术，将模型参数量从70B压缩至7B，推理速度提升10倍，准确率仅下降2.3%。
- 研发了新一代向量检索引擎，支持十亿级向量毫秒级检索，召回率相比传统方案提升15%。
- 构建了多模态文档解析管道，支持PDF、Word、PPT等多种格式。

三、未来展望

2025年将继续深化AI技术在各业务场景的落地应用，重点方向包括Agent智能体、多模态交互、以及端侧推理优化。

文档编号: TECH-2026-001 | 密级: 内部公开 | 日期: 2026-07-24

图2-1: 包含文字内容的图片（模拟扫描件）

2.2 图片中的表格

以下图片包含表格数据。RAG解析器需要能够识别图片中的表格结构，并将其转换为可检索的结构化数据。

表1: 2026年度研发项目汇总表

序号	项目名称	技术栈	负责人	预算(万)	开始日期	结束日期	当前状态
1	大模型训练框架优化	PyTorch, CUDA	张三	500	2026-01-15	2026-06-30	已验收
2	多模态理解系统	Python, CLIP	李四	350	2026-03-01	2026-09-30	进行中
3	RAG检索增强平台	Python, FAISS	王五	280	2026-04-01	2026-12-31	测试中
4	模型压缩工具链	C++, ONNX	赵六	200	2026-05-15	2026-10-31	已验收
5	Agent智能体框架	Python, LangChain	钱七	320	2026-06-01	2027-03-31	开发中
6	向量检索引擎V2	Go, Milvus	孙八	180	2026-07-01	2026-11-30	已验收
7	文档解析管道	Python, OCR	周九	150	2026-08-01	2026-12-15	测试中
8	端侧推理SDK	Rust, ONNX	吴十	400	2026-09-01	2027-06-30	开发中

图2-2: 包含表格的图片

2.3 图片中的折线图

以下图片是一个数据可视化折线图。RAG解析器需要能够感知图表的存在，理想情况下通过多模态模型提取图表中的数据趋势和关键信息。

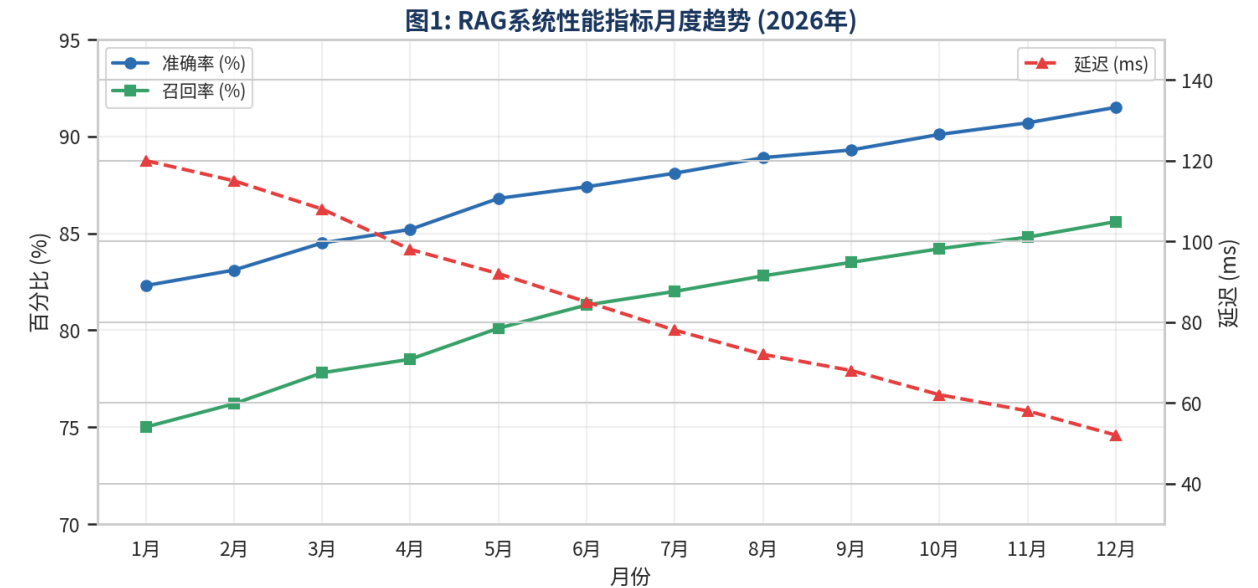


图2-3: RAG系统性能指标月度趋势折线图

2.4 图片中的数学公式

以下图片包含数学公式。RAG解析器需要能够识别公式，通过OCR或MathPix等方式提取LaTeX格式的公式内容。

图2: RAG系统核心数学公式汇总

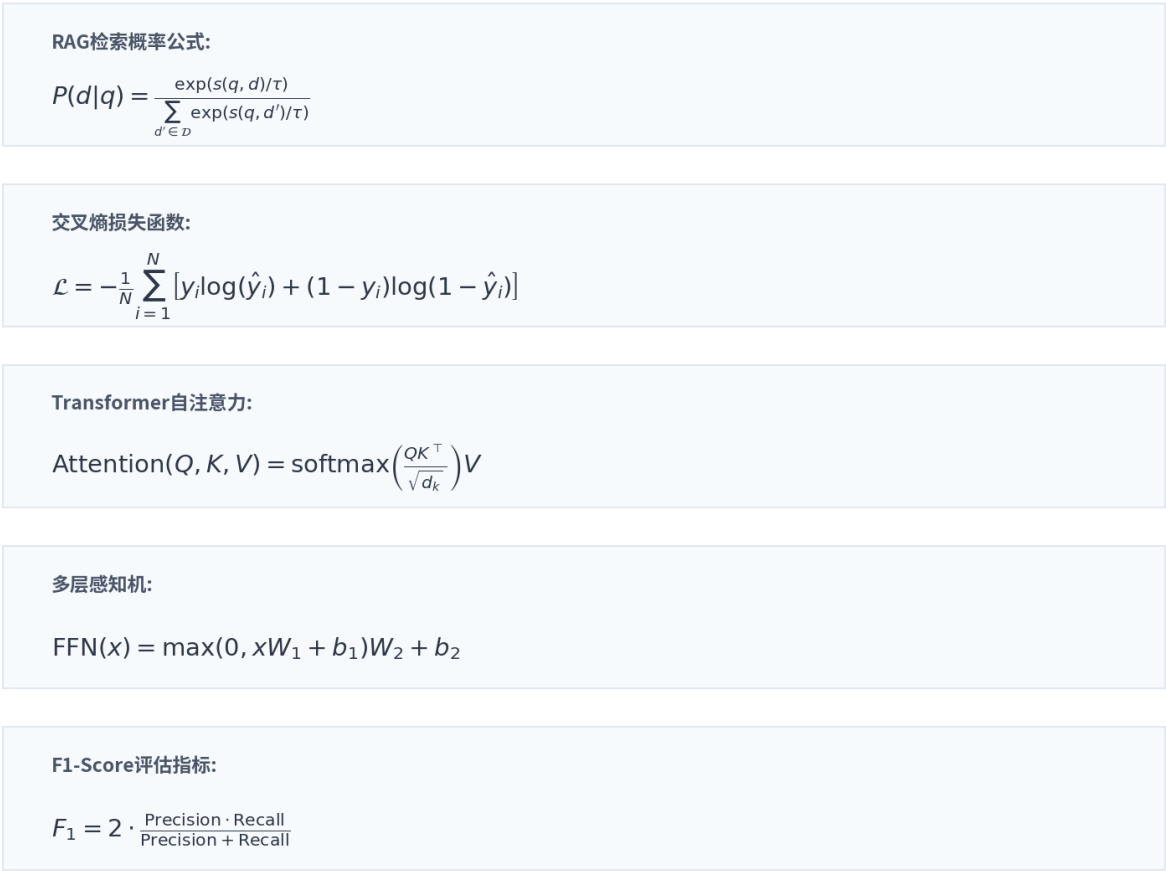


图2-4: RAG系统核心数学公式汇总

2.5 图片中的复杂表格（合并单元格）

以下图片包含带有合并单元格的复杂表格，这种表格结构在PDF解析中容易出错，因为合并单元格会破坏规则的网格结构。

表2: 模型性能对比表 (含合并单元格)

模型名称	参数规模	任务表现	推理速度	内存占用	综合评分	备注
GPT-4o	-	准确率 92.3%	35ms	24GB	9.5	商业API
		召回率 88.7%				
Claude 3.5	-	准确率 91.8%	42ms	20GB	9.3	商业API
		召回率 87.2%				
Llama 3.1 70B	70B	准确率 89.1%	28ms	36GB	9.0	开源模型
		召回率 85.5%				

图2-5: 含合并单元格的复杂表格图片

第三部分：表格测试

本部分测试RAG解析器对不同类型表格的处理能力，包括普通表格、长表格、跨页表格、以及包含合并单元格的复杂表格。

3.1 普通表格

以下是一个标准表格，包含表头、数据行和基本的样式。

序号	姓名	部门	职位	入职日期	绩效评分
1	张三	技术部	高级工程师	2020-03-15	A
2	李四	产品部	产品经理	2019-07-22	A+
3	王五	设计部	UI设计师	2021-01-10	B+
4	赵六	技术部	架构师	2018-11-05	A
5	钱七	市场部	市场总监	2017-06-18	A-

表3-1: 员工基本信息表

3.2 长表格

以下是一个包含30行数据的长表格，测试解析器对大量表格数据的处理能力。

序号	项目名称	负责人	状态	进度(%)	预算(万)	实际支出(万)
1	RAG检索平台	张三	进行中	85	280	245
2	多模态理解	李四	已验收	100	350	338
3	模型压缩工具	王五	测试中	90	200	185
4	Agent框架	赵六	开发中	45	320	120
5	向量引擎V2	钱七	已验收	100	180	175
6	文档解析SDK	孙八	测试中	78	150	130
7	端侧推理引擎	周九	开发中	30	400	95
8	数据标注平台	吴十	规划中	10	500	20
9	知识图谱构建	郑一	进行中	60	220	140
10	对话管理系统	冯二	已验收	100	180	175
11	推荐算法优化	陈三	进行中	70	160	105

序号	项目名称	负责人	状态	进度(%)	预算(万)	实际支出(万)
12	日志分析系统	褚四	已验收	100	120	118
13	权限管理模块	卫五	测试中	88	90	75
14	消息推送服务	蒋六	开发中	50	110	48
15	缓存中间件	沈七	已验收	100	80	76
16	API网关升级	韩八	进行中	65	140	82
17	监控告警系统	杨九	测试中	92	100	88
18	自动化测试框架	朱十	开发中	40	130	42
19	CI/CD管道优化	秦一	已验收	100	75	70
20	容器化部署	尤二	进行中	75	160	115
21	微服务拆分	许三	规划中	15	200	18
22	数据库迁移	何四	进行中	55	180	95
23	安全审计系统	吕五	测试中	82	150	128
24	备份恢复方案	施六	已验收	100	95	90
25	负载均衡优化	张七	开发中	35	120	38
26	DNS服务改造	孔八	规划中	5	60	3
27	邮件服务升级	曹九	已验收	100	70	65
28	搜索服务优化	严十	进行中	68	130	82
29	配置中心建设	华一	测试中	87	85	72
30	服务网格部署	金二	开发中	42	250	88

表3-2: 2026年度研发项目汇总表（30项）

3.3 跨页面表格

以下是一个包含50行数据的大型表格，必然会跨越多页。测试解析器对跨页表格的处理，特别是表头重复和页面分割处的数据完整性。

序号	日期	操作类型	操作用户	IP地址	操作详情	结果
1	2026-02-19	查询	wangwu	192.168.31.127	执行了提交操作，涉及数据表 logs	部分成功

序号	日期	操作类型	操作用户	IP地址	操作详情	结果
2	2026-07-28	登录	zhangsan	192.168.22.93	执行了导出操作, 涉及数据表 projects	失败
3	2026-02-19	审核	lisi	192.168.95.235	执行了发布操作, 涉及数据表 logs	成功
4	2026-02-10	查询	zhaoliu	192.168.123.40	执行了查询操作, 涉及数据表 users	成功
5	2026-05-09	导入	qianqi	192.168.8.216	执行了删除操作, 涉及数据表 logs	部分成功
6	2026-07-19	查询	zhaoliu	192.168.196.67	执行了登录操作, 涉及数据表 users	成功
7	2026-03-05	审批	zhangsan	192.168.19.211	执行了导入操作, 涉及数据表 users	部分成功
8	2026-02-12	审批	lisi	192.168.50.181	执行了查询操作, 涉及数据表 users	成功
9	2026-04-20	导入	wangwu	192.168.36.48	执行了导入操作, 涉及数据表 users	部分成功
10	2026-05-02	审批	zhaoliu	192.168.148.4	执行了提交操作, 涉及数据表 projects	失败
11	2026-04-28	审核	zhaoliu	192.168.116.131	执行了修改操作, 涉及数据表 projects	部分成功
12	2026-04-09	审核	wangwu	192.168.37.98	执行了导入操作, 涉及数据表 users	失败
13	2026-03-10	修改	lisi	192.168.171.38	执行了修改操作, 涉及数据表 projects	成功
14	2026-01-20	审核	lisi	192.168.70.189	执行了导出操作, 涉及数据表 documents	部分成功
15	2026-02-01	审核	zhangsan	192.168.41.234	执行了登录操作, 涉及数据表 projects	成功
16	2026-03-16	提交	wangwu	192.168.107.59	执行了登录操作, 涉及数据表 logs	失败
17	2026-05-14	导出	wangwu	192.168.75.45	执行了查询操作, 涉及数据表 documents	成功
18	2026-02-04	提交	wangwu	192.168.62.51	执行了提交操作, 涉及数据表 logs	成功

序号	日期	操作类型	操作用户	IP地址	操作详情	结果
19	2026-06-02	审核	zhangsan	192.168.131.52	执行了导入操作, 涉及数据表 logs	部分成功
20	2026-06-22	修改	wangwu	192.168.173.163	执行了导入操作, 涉及数据表 users	部分成功
21	2026-02-14	审批	wangwu	192.168.123.216	执行了提交操作, 涉及数据表 projects	部分成功
22	2026-02-08	登录	qianqi	192.168.115.48	执行了导入操作, 涉及数据表 users	成功
23	2026-07-10	查询	zhangsan	192.168.152.79	执行了审批操作, 涉及数据表 logs	部分成功
24	2026-06-13	发布	lisi	192.168.56.43	执行了修改操作, 涉及数据表 projects	部分成功
25	2026-04-25	登录	lisi	192.168.239.224	执行了提交操作, 涉及数据表 documents	成功
26	2026-06-07	删除	zhaoliu	192.168.154.140	执行了修改操作, 涉及数据表 users	失败
27	2026-06-16	导出	wangwu	192.168.51.19	执行了导出操作, 涉及数据表 users	部分成功
28	2026-01-24	修改	qianqi	192.168.153.183	执行了删除操作, 涉及数据表 projects	成功
29	2026-05-03	导入	qianqi	192.168.217.156	执行了查询操作, 涉及数据表 documents	部分成功
30	2026-05-26	导出	qianqi	192.168.8.160	执行了导出操作, 涉及数据表 projects	成功
31	2026-01-16	查询	zhaoliu	192.168.37.176	执行了修改操作, 涉及数据表 users	成功
32	2026-01-07	修改	wangwu	192.168.179.231	执行了提交操作, 涉及数据表 projects	成功
33	2026-05-06	删除	wangwu	192.168.123.167	执行了发布操作, 涉及数据表 projects	部分成功
34	2026-05-12	发布	zhangsan	192.168.216.59	执行了审批操作, 涉及数据表 users	失败
35	2026-05-11	修改	wangwu	192.168.222.40	执行了审批操作, 涉及数据表 documents	失败

序号	日期	操作类型	操作用户	IP地址	操作详情	结果
36	2026-03-02	登录	wangwu	192.168.225.21	执行了审核操作, 涉及数据表 users	部分成功
37	2026-06-20	查询	qianqi	192.168.171.216	执行了导入操作, 涉及数据表 users	失败
38	2026-06-14	审批	zhangsan	192.168.17.94	执行了导入操作, 涉及数据表 logs	成功
39	2026-02-09	审批	zhangsan	192.168.50.57	执行了发布操作, 涉及数据表 logs	成功
40	2026-04-19	审核	zhangsan	192.168.63.73	执行了修改操作, 涉及数据表 projects	失败
41	2026-04-11	删除	lisi	192.168.231.234	执行了发布操作, 涉及数据表 logs	部分成功
42	2026-06-04	登录	wangwu	192.168.63.58	执行了查询操作, 涉及数据表 projects	部分成功
43	2026-01-10	修改	zhaoliu	192.168.95.172	执行了提交操作, 涉及数据表 users	部分成功
44	2026-04-17	审核	zhangsan	192.168.46.98	执行了导出操作, 涉及数据表 projects	成功
45	2026-03-14	提交	wangwu	192.168.21.78	执行了登录操作, 涉及数据表 documents	成功
46	2026-01-13	登录	qianqi	192.168.151.221	执行了审核操作, 涉及数据表 projects	成功
47	2026-03-03	审批	qianqi	192.168.117.147	执行了导出操作, 涉及数据表 documents	部分成功
48	2026-05-11	修改	qianqi	192.168.183.137	执行了审核操作, 涉及数据表 documents	成功
49	2026-01-12	审批	zhangsan	192.168.83.193	执行了登录操作, 涉及数据表 users	部分成功
50	2026-04-28	导出	wangwu	192.168.164.229	执行了提交操作, 涉及数据表 projects	失败

表3-3: 系统操作日志表 (50条记录, 跨页展示)

3.4 合并单元格表格

以下表格包含合并单元格。RAG解析器需要正确处理合并单元格，避免数据错位。

大分类	子分类	Q1	Q2	Q3	Q4
AI产品	智能客服	120	135	148	162
	文档分析	85	92	105	118
	代码助手	60	72	88	95
云服务	对象存储	200	215	230	248
	CDN加速	150	158	165	178
	容器服务	95	108	120	135
数据服务	数据仓库	180	192	205	220
	实时计算	110	125	138	152

表3-4: 季度营收汇总表（含合并单元格）

3.5 长表格（表头仅在第一页，跨页不重复）

以下是一个多列长表格，表头行仅在第一页出现，跨页后续页面不重复表头。这种场景用于测试RAG解析器能否正确识别跨页表格的连续性，即使后续页面没有表头，也应能将数据正确关联到表头字段。

序号	命令	说明	示例
1	django-admin startproject	创建新项目	django-admin startproject myproject
2	python manage.py startapp	创建新应用	python manage.py startapp blog
3	python manage.py runserver	启动开发服务器	python manage.py runserver 0.0.0.0:8000
4	python manage.py makemigrations	生成迁移文件	python manage.py makemigrations blog
5	python manage.py migrate	执行数据库迁移	python manage.py migrate
6	python manage.py createsuperuser	创建超级用户	python manage.py createsuperuser
7	python manage.py shell	进入Django Shell	python manage.py shell
8	python manage.py dbshell	进入数据库Shell	python manage.py dbshell
9	python manage.py test	运行测试	python manage.py test blog.tests
10	python manage.py collectstatic	收集静态文件	python manage.py collectstatic --noinput
11	python manage.py flush	清空数据库	python manage.py flush --noinput
12	python manage.py loaddata	加载fixture数据	python manage.py loaddata initial_data.json
13	python manage.py dumpdata	导出fixture数据	python manage.py dumpdata blog.Article --indent=2
14	python manage.py inspectdb	反向生成Model	python manage.py inspectdb > models.py

15	python manage.py changepassword	修改用户密码	python manage.py changepassword admin
16	python manage.py createcachetable	创建缓存表	python manage.py createcachetable
17	python manage.py sendtestemail	发送测试邮件	python manage.py sendtestemail test@example.com
18	python manage.py compilemessages	编译国际化文件	python manage.py compilemessages
19	python manage.py makemessages	生成国际化文件	python manage.py makemessages -l zh_Hans
20	python manage.py squashmigrations	合并迁移	python manage.py squashmigrations blog 0001 0010
21	python manage.py showmigrations	查看迁移状态	python manage.py showmigrations blog
22	python manage.py sqlmigrate	查看迁移SQL	python manage.py sqlmigrate blog 0001
23	python manage.py sqlsequencereset	重置序列SQL	python manage.py sqlsequencereset blog
24	python manage.py check	项目检查	python manage.py check --deploy
25	python manage.py optimizemigration	优化迁移	python manage.py optimizemigration blog 0001
26	django-admin version	查看Django版本	django-admin version
27	python manage.py remove_stale_contenttypes	清理过期ContentType	python manage.py remove_stale_contenttypes
28	python manage.py clearsessions	清理过期会话	python manage.py clearsessions
29	python -m pip install django	安装Django	python -m pip install django==4.2
30	python -m pip install djangorestframework	安装DRF	python -m pip install djangorestframework
31	python -m pip install django-cors-headers	安装CORS扩展	python -m pip install django-cors-headers
32	python -m pip install celery	安装Celery	python -m pip install celery[redis]
33	python -m pip install django-debug-toolbar	安装调试工具栏	python -m pip install django-debug-toolbar
34	python -m pip install django-extensions	安装扩展命令	python -m pip install django-extensions
35	python -m pip install gunicorn	安装Gunicorn	python -m pip install gunicorn
36	python -m pip install uvicorn	安装Uvicorn	python -m pip install uvicorn[standard]
37	python -m pip install whitenoise	安装静态文件服务	python -m pip install whitenoise
38	python -m pip install redis	安装Redis客户端	python -m pip install redis
39	python -m pip install psycopg2-binary	安装PostgreSQL驱动	python -m pip install psycopg2-binary
40	python -m pip install mysqlclient	安装MySQL驱动	python -m pip install mysqlclient

表3-5: Django常用命令速查表（40条命令，表头仅在第一页，跨页不重复）

第四部分：文本格式与样式测试

4.1 富文本格式测试

以下段落包含粗体文字、斜体文字、下划线文字、蓝色文字、红色文字、**大号文字**、小号文字。还有上标 $E=mc^2$ 和下标 H_2O 化学式。还有 ~~删除线文字~~ 和 **粗斜体加下划线**。

4.2 超链接测试

本文档参考了以下资料：

RAG技术论文：<https://arxiv.org/abs/2005.11401>

OpenAI官方文档：<https://platform.openai.com/docs>

LangChain框架：<https://www.langchain.com>

4.3 列表格式测试

无序列表（Unordered List）：

- Django采用MVT架构，Model负责数据层，View负责逻辑层，Template负责展示层
- Django ORM支持链式查询，QuerySet采用延迟加载机制提升性能
- Django admin后台可根据Model自动生成CRUD管理界面
- Django REST Framework是构建RESTful API的事实标准扩展

有序列表（Ordered List）：

1. 使用django-admin startproject创建项目
2. 使用python manage.py startapp创建应用
3. 在models.py中定义数据模型
4. 执行makemigrations和migrate生成数据库表
5. 在views.py中编写视图逻辑并配置urls.py路由

嵌套列表（Nested List）：

1. Model层（数据模型）
 - 1.1 ORM查询
 - 1.1.1 select_related（JOIN预加载）
 - 1.1.2 prefetch_related（二次查询预加载）
 - 1.2 数据库迁移
2. View层（视图逻辑）

2.1 函数视图 (FBV)

2.2 类视图 (CBV)

4.4 引用块测试

Django官方文档中提到：

"Django was designed to help developers take applications from concept to completion as quickly as possible. It takes security seriously and helps developers avoid many common security mistakes."

Django框架的设计哲学强调快速开发、DRY原则和安全性。它内置了CSRF防护、XSS过滤、SQL注入防护、密码哈希等安全机制，开发者只需遵循最佳实践即可构建安全的Web应用。

4.5 代码块测试

以下是一个Django models.py的代码示例，测试RAG解析器对代码块的处理：

```
```python
from django.db import models
from django.contrib.auth.models import User

class Article(models.Model):
 """文章模型"""
 title = models.CharField(max_length=200, verbose_name='标题')
 content = models.TextField(verbose_name='正文')
 author = models.ForeignKey(User, on_delete=models.CASCADE, related_name='articles')
 category = models.ForeignKey('Category', on_delete=models.SET_NULL, null=True)
 tags = models.ManyToManyField('Tag', blank=True)
 views = models.PositiveIntegerField(default=0)
 is_published = models.BooleanField(default=False)
 created_at = models.DateTimeField(auto_now_add=True)
 updated_at = models.DateTimeField(auto_now=True)

 class Meta:
 ordering = ['-created_at']
 verbose_name = '文章'
 verbose_name_plural = verbose_name

 def __str__(self):
 return self.title

 def increment_views(self):
 self.views = models.F('views') + 1
 self.save(update_fields=['views'])
```
```

注意：RAG解析器需要正确处理代码块，保留缩进和换行，避免将代码与正文混淆。上面的代码展示了Django Model的常见字段类型：CharField、TextField、ForeignKey、ManyToManyField、DateTimeField等。

4.6 纯英文段落测试

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. It follows the MVT (Model-View-Template) architectural pattern and emphasizes the DRY (Don't Repeat Yourself) principle. Built by experienced developers, Django takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It is free and open source, with a thriving community and excellent documentation. Django includes built-in features like an ORM, authentication system, admin interface, URL routing, template engine, form handling, and caching framework. The Django REST Framework (DRF) extends Django to build RESTful APIs with serializers, viewsets, routers, and authentication. Django's ORM supports multiple database backends including PostgreSQL, MySQL, SQLite, and Oracle, and provides powerful query optimization tools like `select_related` and `prefetch_related`.

4.7 不同字号对比

这是正常字号 (11pt) 的文本。

这是大字号 (16pt) 的文本，用于测试大字体解析。

这是小字号 (8pt) 的文本，用于测试极小字体是否会被忽略。某些PDF文档中的脚注和注释会使用非常小的字号，RAG解析器需要能够正确识别这些内容。

4.8 文本对齐方式

左对齐文本：检索增强生成（Retrieval-Augmented Generation，简称RAG）是一种将信息检索与文本生成相结合的人工智能技术架构。

居中对齐文本：检索增强生成（Retrieval-Augmented Generation，简称RAG）是一种将信息检索与文本生成相结合的人工智能技术架构。

右对齐文本：检索增强生成（Retrieval-Augmented Generation，简称RAG）是一种将信息检索与文本生成相结合的人工智能技术架构。

两端对齐文本（Justify）：检索增强生成（Retrieval-Augmented Generation，简称RAG）是一种将信息检索与文本生成相结合的人工智能技术架构。在传统的生成式AI模型中，模型仅依赖其训练阶段学到的知识来回答问题，这导致模型在面对需要最新信息或特定领域知识的问题时，往往表现不佳。RAG技术通过在生成答案之前先从外部知识库中检索相关信息，并将检索到的信息作为上下文注入到生成过程中，从而显著提升了模型回答的准确性、时效性和可解释性。

第五部分：特殊元素与边界情况测试

5.1 公章（印章）测试

以下图片是一个模拟的公章。在正式文档中，公章通常出现在文档末尾或特定位置。RAG解析器需要能够识别公章的存在，并在提取信息时标注公章的位置。



图5-1: 模拟公章

5.2 页眉页脚与页码

本文档每一页都包含页眉（"RAG解析验证测试文档"和版本号）和页脚（页码）。RAG解析器应当能够区分正文内容与页眉页脚，避免将页眉页脚的内容混入正文检索。同时，页码信息应当被正确识别，用于文档内容定位。

5.3 水印测试

本文档每一页背景都包含"RAG 测试文档"和"内部资料"的水印文字，呈45度角倾斜排列。RAG解析器应能够识别水印的存在，并避免将水印文字作为正文内容进行检索。水印通常具有以下特征：颜色较浅、位于页面背景层、文字倾斜排列。

5.4 表格中的特殊字符

| 字符类型 | 示例 | 描述 |
|------|--------------------------------|------|
| 数学符号 | $\pm 50\%$ | 正负号 |
| 货币符号 | ¥ 1,280,000 | 人民币 |
| 百分比 | 98.5‰ | 千分号 |
| 温度单位 | 37.5°C | 摄氏度 |
| 化学式 | H ₂ SO ₄ | 硫酸 |
| 上下标 | $x^2 + y^3 = z^4$ | 指数 |
| 希腊字母 | $\alpha=0.05, \beta=0.15$ | 统计参数 |

| | | |
|----|-----------|------|
| 箭头 | A → B → C | 流程方向 |
|----|-----------|------|

表5-1: 特殊字符在表格中的展示

5.5 空表格（仅有表头）

| 序号 | 姓名 | 部门 | 备注 |
|----|----|----|----|
|----|----|----|----|

表5-2: 空表格（仅有表头，无数据行）

注意：空表格是RAG解析中的边界情况。解析器应能正确处理仅有表头无数据的表格。

5.6 单单元格表格

这是一个仅包含一个单元格的表格，用于测试解析器对最小表格结构的处理能力。

表5-3: 单单元格表格

5.7 包含空单元格的表格

| 姓名 | 电话 | 邮箱 | 地址 |
|----|-------------|------------------|---------|
| 张三 | 13800138000 | | 北京市海淀区 |
| 李四 | | lisi@example.com | |
| 王五 | | | 上海市浦东新区 |
| | 13900139000 | zhao@example.com | 深圳市南山区 |

表5-4: 包含空单元格的表格

注意：空单元格是常见的边界情况。RAG解析器应正确处理空单元格，不应将空值误认为解析错误或跳过该单元格。

5.8 超长单元格文本

| 序号 | 问题描述 | 解决方案 |
|----|--------------|--|
| 1 | PDF解析时出现乱码字符 | 使用pdfplumber替代pypdf进行文本提取，并配置正确的编码格式（UTF-8）。对于无法自动 |
| 2 | 表格识别精度不足 | 结合多种表格检测算法（规则检测+深度学习），对无边框表格使用启发式规则，对有边框表 |
| 3 | 公式无法正确解析 | 集成MathPix API进行公式OCR识别，将识别结果转换为LaTeX格式。对于内嵌公式使用MathT |

表5-5: 包含超长文本单元格的表格

5.9 脚注与注释

RAG系统的性能评估通常使用以下指标：准确率（Accuracy）、召回率（Recall）、F1分数（F1-Score）和平均倒数排名（MRR）¹。其中，F1分数是精确率与召回率的调和平均数²，能够综合反映系统的检索质量³。

¹ 参考：Manning, C. D., et al. "Introduction to Information Retrieval." Cambridge University Press, 2008.

² 调和平均数的定义为 $H = n / (1/x_1 + 1/x_2 + \dots + 1/x_n)$ 。

³ 在RAG场景中，检索质量直接影响最终答案的准确性，因此检索环节的F1分数是系统性能的关键指标。

5.10 高亮/重点标注

【重要提示】 本文档中的所有测试用例均基于RAG系统在实际应用中可能遇到的真实场景构建。请确保您的RAG解析器能够正确处理本文档中列出的每一种元素类型，以避免在生产环境中出现解析失败、数据丢失或检索错误等问题。

5.11 异形文本排版

以下展示一些不常见的文本排版方式：

连续标点：！！！！！！？？？？？。。。。。，，，，，，、、、、、、

超长英文单词：Pneumonoultramicroscopicsilicovolcanoconiosis
Hippopotomonstrosesquippedaliophobia Supercalifragilisticexpialidocious

无空格连续中文英文混合：RAG是Retrieval增强生成的缩写它结合了InformationRetrieval和TextGeneration

附录A：RAG解析器测试检查清单

短段落正确识别，不会合并

长段落完整保留，不会截断

空白段落正确处理

多级标题层级正确解析（4层以上）

中英文混合文本正确提取

特殊Unicode字符正确保留

图片中的文字通过OCR正确提取

图片中的表格正确识别为结构化数据

图片中的图表正确标注

图片中的公式正确转换为LaTeX

普通表格正确解析

长表格完整提取

跨页表格正确处理（表头重复、数据完整）

合并单元格正确解析

空表格/空单元格正确处理

页眉页脚与正文正确区分

水印不被混入正文

公章/印章正确识别或标注

页码正确识别

粗体/斜体/下划线样式保留

上标/下标正确转换

超链接正确提取

有序/无序列表正确解析

嵌套列表层级正确

引用块正确识别

代码块格式保留

不同字号文本正确解析

不同对齐方式正确保留

脚注正确关联

超长单元格文本完整提取

特殊字符在表格中正确显示

连续标点/超长单词正确处理